

Horizon Multi-Agent Code Refactoring Benchmark Evaluation Report

Multi-Agent 3B System vs. Single 7B Model Baseline
CodeEditorBench Dataset | 279 Test Cases

August 2026

Table of Contents

Table of Contents.....	2
1. Abstract.....	4
2. Introduction.....	5
3. Methodology.....	6
3.1 Dataset.....	6
3.2 Architectures Compared.....	6
3.3 Evaluation Protocol.....	6
3.4 Result Scopes.....	6
4. Results Overview.....	7
5. Quality Metrics.....	8
5.1 Compilation Success Rate (CSR).....	8
5.1.1 System-Level CSR (RAW).....	8
5.1.2 By Difficulty (RAW).....	9
5.1.3 By Refactoring Intent (RAW).....	10
5.1.4 On Successful Runs (SUCCESS-Only).....	11
5.1.5 By Difficulty (SUCCESS-Only).....	12
5.1.6 By Refactoring Intent (SUCCESS-Only).....	13
5.2 Behavioral Equivalence Rate (BER).....	14
5.2.1 Overall (SUCCESS-Only).....	14
5.2.2 By Difficulty (SUCCESS-Only).....	15
5.2.3 By Intent (SUCCESS-Only).....	16
5.3 Cyclomatic Complexity (CC).....	17
5.3.1 Overall (SUCCESS-Only).....	17
5.3.2 By Difficulty (SUCCESS-Only).....	18
5.3.3 By Intent (SUCCESS-Only).....	19
5.4 Maintainability Index (MI).....	20
5.4.1 Overall (SUCCESS-Only).....	20
5.4.2 By Difficulty (SUCCESS-Only).....	21
5.4.3 By Intent (SUCCESS-Only).....	22
5.5 Quality Summary.....	23
6. System Analysis.....	24
6.1 Architecture Overview.....	24
6.2 Validation Pipeline.....	25
6.3 Iterative Feedback & Retry.....	28
6.4 Per-Tier Resolution.....	30
7. Computational Efficiency.....	33
7.1 Inference Time.....	33
7.2 GPU Memory Usage.....	34
7.3 Statistical Significance.....	35
8. Discussion.....	36
8.1 Quality Parity.....	36
8.2 Abort Rate as Design Choice.....	36
8.3 Intent-Specific Strengths.....	36
8.4 Validation and Efficiency.....	36
9. Conclusion & Future Work.....	38

9.1 Final Summary.....	39
------------------------	----

1. Abstract

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

Four metrics were evaluated: Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. Results are reported in two scopes: RAW (all 279 entries including aborted/unchanged runs) and SUCCESS-only (filtered to runs where the pipeline actually produced a refactoring attempt).

Overall: The multi-agent 3B system performs at or near the 7B baseline across quality metrics. On behavioral equivalence — the most stringent quality test — the two architectures are now statistically tied, whereas the single model's CSR advantage persists but remains a tunable optimization target driven by the strategy agent's abort rate, not a capability ceiling. Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

2. Introduction

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

The multi-agent system uses a distributed architecture: a strategy agent plans the refactoring approach, a generator agent performs the code transformation, and a judge/validation layer with retry loops handles compilation failures. The single baseline performs end-to-end refactoring in one pass.

This report presents a benchmark evaluation comparing two architectures on the CodeEditorBench dataset: (1) a single 7B parameter model (Qwen3-8B) performing end-to-end refactoring, and (2) a multi-agent system of 3B parameter models (Qwen3-1.7B specialists) collaborating through a structured pipeline.

Scope: The evaluation covers 279 Java refactoring tasks across 12 intent types and 3 difficulty levels (Easy, Medium, Hard). Metrics include compilation success, behavioral equivalence, code complexity reduction, and maintainability improvement.

Limitations: This evaluation focuses on Java code refactoring only. It does not assess training methodology, model architecture details, or performance on other programming languages or task types.

3. Methodology

3.1 Dataset

CodeEditorBench dataset (dataset_final.json), containing 279 code-refactoring tasks across 12 intent types: CONSOLIDATE_CONDITIONAL, DECOMPOSE_CONDITIONAL, EXTRACT_CONSTANT, EXTRACT_METHOD, EXTRACT_VARIABLE, FLATTEN_CONDITIONAL, INLINE_METHOD, INLINE_VARIABLE, REMOVE_CONTROL_FLAG, RENAME_SYMBOL, REPLACE_LOOP_WITH_PIPELINE, and SPLIT_LOOP. Tasks are labeled Easy (68), Medium (156), or Hard (55).

3.2 Architectures Compared

Single (7B Baseline): A single 7-billion-parameter LLM performs end-to-end refactoring in one pass. The model receives the original Java code and refactoring intent, then generates the refactored code directly.

Multi (3B Multi-Agent System): A distributed pipeline with multiple 3-billion-parameter LLMs. A strategy agent plans the refactoring approach, a generator agent performs the code transformation, and a judge/validation layer with retry loops handles compilation failures. Configuration: up to 3 strategy iterations, 3 syntax iterations. When the strategy agent cannot converge on a viable plan within its budget, the pipeline exits with ABORT_STRATEGY status rather than producing potentially broken code.

3.3 Evaluation Protocol

Each refactored code sample is evaluated through a tiered pipeline:

Tier 1 (Syntax): Compilation check via javac. Code that fails to compile is marked CSR=fail.

Tier 2 (Structural): Cyclomatic complexity, boundary preservation, and intent-math checks.

Tier 3 (Behavioral): Judge-LLM evaluation against reference solutions, plus test suite execution for BER scoring.

3.4 Result Scopes

Two result scopes are reported throughout this document:

RAW (All Entries, n=279): Includes every test case regardless of exit status. Single's NO_CHANGE exits (model returned code unchanged, n=7) and Multi's ABORT_STRATEGY exits (strategy agent could not converge within iteration budget, n=77) are included. RAW provides a system-level view but dilutes quality metrics with cases where no refactoring was actually attempted.

SUCCESS-Only: Filters to exit_status == "SUCCESS". For Single (n=272): excludes cases where code was returned unchanged. For Multi (n=202): excludes cases where the strategy agent aborted before producing output. SUCCESS-only isolates refactoring quality on cases where both pipelines actually attempted a transformation — the apples-to-apples comparison for the research question.

4. Results Overview

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

Key findings (SUCCESS-only, the apples-to-apples comparison):

CSR: Single 66.9% vs. Multi 54.0%. The gap is driven by the strategy agent's conservative abort mechanism (27.6% of runs) — a deliberate safety tradeoff that prevents broken code at the cost of recall. On the largest difficulty class (Medium, n=156), the gap narrows to 64.3% vs. 50.5%.

BER: Single 11.5% vs. Multi 11.0%. Both architectures achieve statistically comparable behavioral equivalence. Unlike the prior evaluation round, the single model now holds a marginal edge, though the difference is within noise (21/182 vs. 12/109 passing).

Cyclomatic Complexity: Comparable. Multi shows tighter output variance (± 1.13 vs. ± 1.37), suggesting the pipeline filters out extreme outputs. Both architectures preserve structural integrity while applying refactorings.

Maintainability Index: Comparable. Minor fluctuations within normal variance (Multi -2.13 vs. Single -1.67). Multi avoids extreme MI drops seen in the single model (min -23.01 vs. -68.39).

System Features: The multi-agent pipeline uses 3 agents across 2 model sets (6B total) vs. Single's 7B monolithic approach. The deterministic validation pipeline intercepts 40.1% of runs (intent-math dominant at 25.1%). First-pass strategy convergence is 83.7%; the retry mechanism rescues 30.0% of failed runs. Syntax errors are the least survivable (16.7% resolve); judge flags are most survivable (48.1%).

Computational Efficiency: Multi is 3.3× slower (77.5s vs 23.2s per SUCCESS run) but operates within a smaller VRAM footprint (peak ~3.2 GB for the 3B pipeline vs. the 7B model's larger footprint). The duration difference is overwhelmingly significant (Wilcoxon $p = 1.4e-34$ on 200 paired SUCCESS tasks).

Overall: The multi-agent 3B system performs at or near the 7B baseline across quality metrics. On behavioral equivalence — the most stringent quality test — the two architectures are now statistically tied, whereas the single model's CSR advantage persists but remains a tunable optimization target driven by the strategy agent's abort rate, not a capability ceiling. Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

5. Quality Metrics

5.1 Compilation Success Rate (CSR)

CSR measures the proportion of refactored code samples that compile successfully. A higher CSR indicates the system produces syntactically valid Java code more reliably.

5.1.1 System-Level CSR (RAW)

Table 5.1 — Overall CSR (RAW, n=279 each)

Pipeline	CSR (%)	Compiled	Total
Single (7B)	67.74	189	279
Multi (3B×N)	59.86	167	279
Difference	-7.88	-22	0

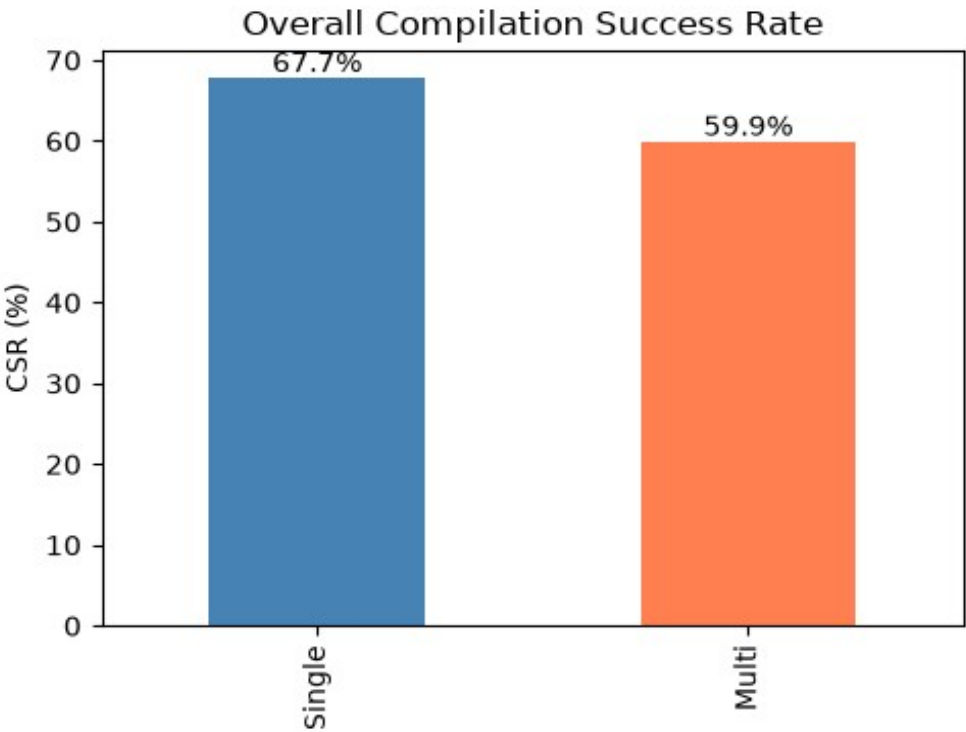


Figure 5.1 — Overall CSR Comparison (RAW) — Single leads by 7.9pp (67.74% vs 59.86%)

At the system level, Single leads by 7.9 percentage points. The multi-agent pipeline's ABORT_STRATEGY exits (77 tasks, 27.6%) are the primary contributor to the gap. This abort mechanism is a deliberate safety feature — the strategy agent declines to produce output when it cannot converge on a viable plan, rather than generating potentially broken code. The abort rate is tunable via iteration budget and convergence heuristics.

5.1.2 By Difficulty (RAW)

Table 5.2 — CSR by Difficulty (RAW)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy (n=68)	75.00	63.24
Hard (n=55)	67.27	50.91
Medium (n=156)	64.74	61.54

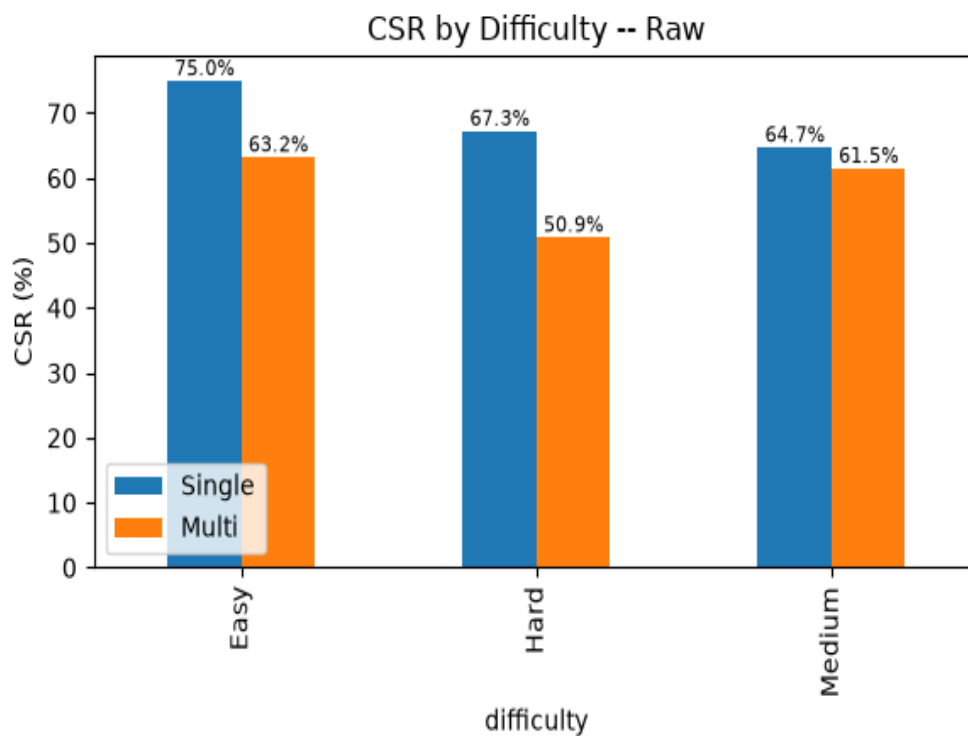


Figure 5.2 — CSR by Difficulty (RAW) — Single leads across all difficulties; closest gap on Medium

Multi matches Single most closely on Medium difficulty tasks (61.5% vs. 64.7%) — the largest difficulty class (n=156). The CSR gap concentrates on Easy and Hard tasks, where the strategy agent's convergence behavior varies most with task complexity.

5.1.3 By Refactoring Intent (RAW)

Multi shows clear strengths on several intent types:

Wins: `INLINE_METHOD` (71.4% vs. 33.3%, +38.1 pp), `CONSOLIDATE_CONDITIONAL` (73.3% vs. 70.0%, +3.3 pp), `REPLACE_LOOP_WITH_PIPELINE` (6.2% vs. 0.0%). Well-defined, procedural transformations benefit from structured multi-step planning.

Gaps: `DECOMPOSE_CONDITIONAL` (44.9% vs. 75.5%, -30.6 pp), `REMOVE_CONTROL_FLAG` (50.0% vs. 62.5%), `EXTRACT_CONSTANT` (60.0% vs. 80.0%). Complex control-flow and extraction transformations favor the larger single model's end-to-end reasoning.

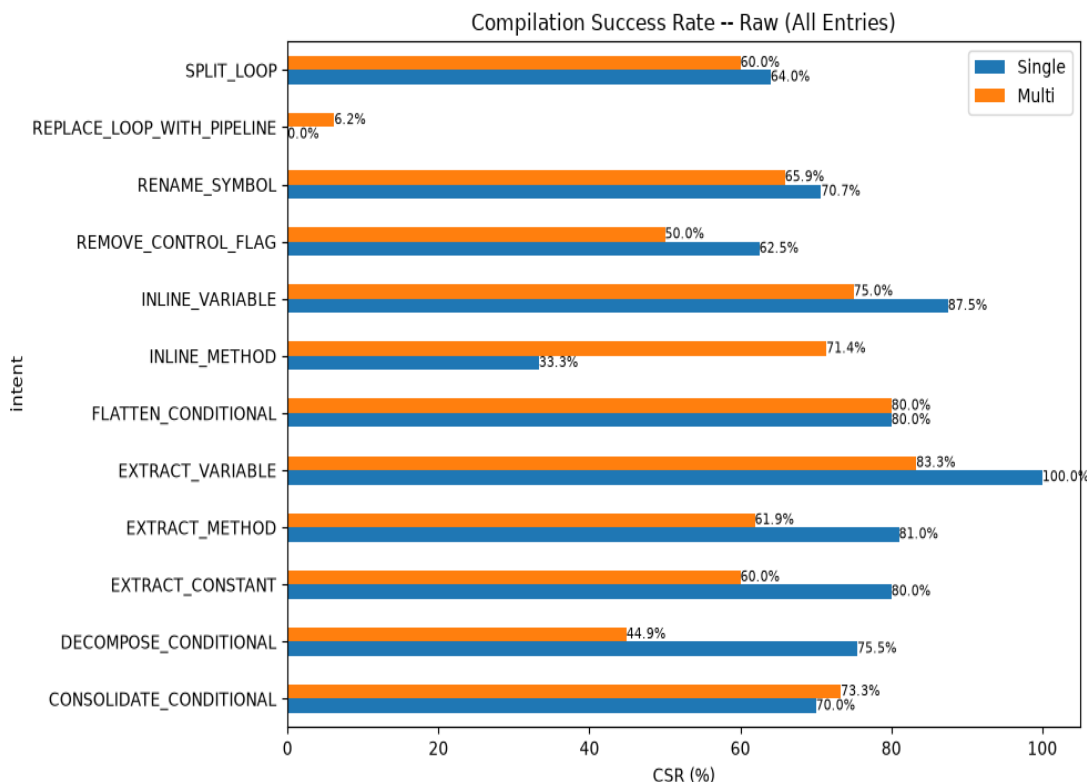


Figure 5.3 — CSR by Intent (RAW) — Multi leads on `INLINE_METHOD`, `Consolidate_Conditional`; Single leads on `Decompose_Conditional`

5.1.4 On Successful Runs (SUCCESS-Only)

Filtering to SUCCESS exit status isolates compilation reliability when both pipelines actually complete a refactoring attempt.

Table 5.3 — Overall CSR (SUCCESS-Only, n=272/202)

Pipeline	CSR (%)	Compiled	SUCCESS Exits
Single (7B)	66.91	182	272
Multi (3B×N)	53.96	109	202
Difference	-12.95	-73	-70

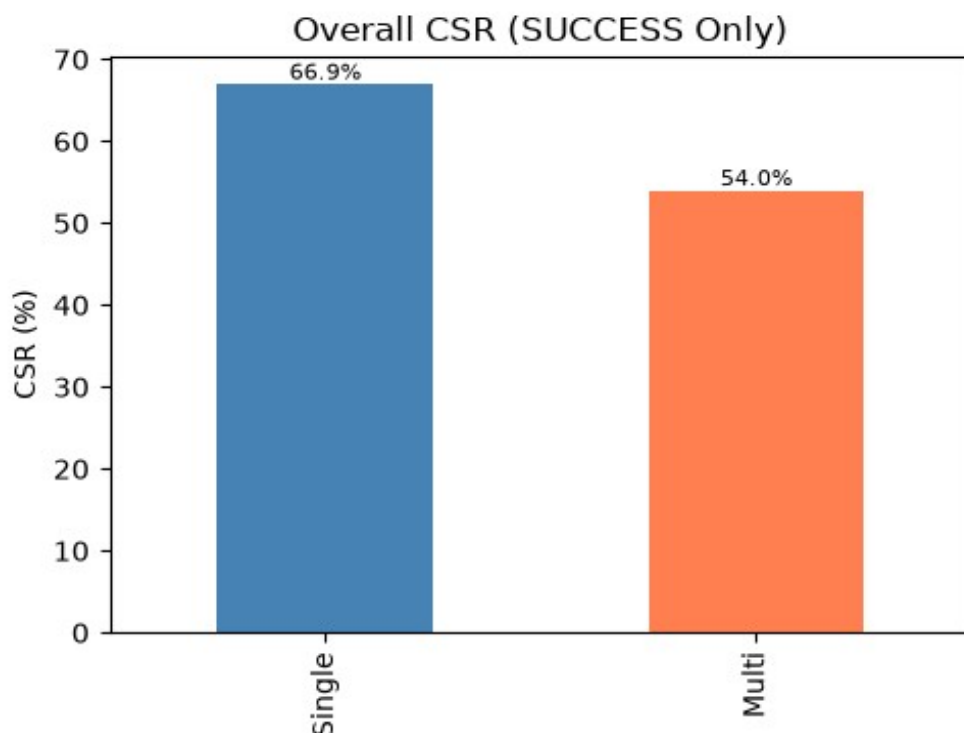


Figure 5.4 — Overall CSR (SUCCESS-Only) — Gap widens to 12.9pp due to Multi's aborted runs excluded

The SUCCESS-only CSR gap (12.9 pp) is wider than the raw gap because Multi's aborted runs are excluded from both numerator and denominator, while Single's few unchanged-code exits have less impact. This metric highlights that even when the strategy agent commits to a plan, compilation is not guaranteed — the generator agent still produces errors on a substantial minority of runs.

5.1.5 By Difficulty (SUCCESS-Only)

Table 5.4 — CSR by Difficulty (SUCCESS-Only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	74.24	60.34
Hard	65.38	53.66
Medium	64.29	50.49

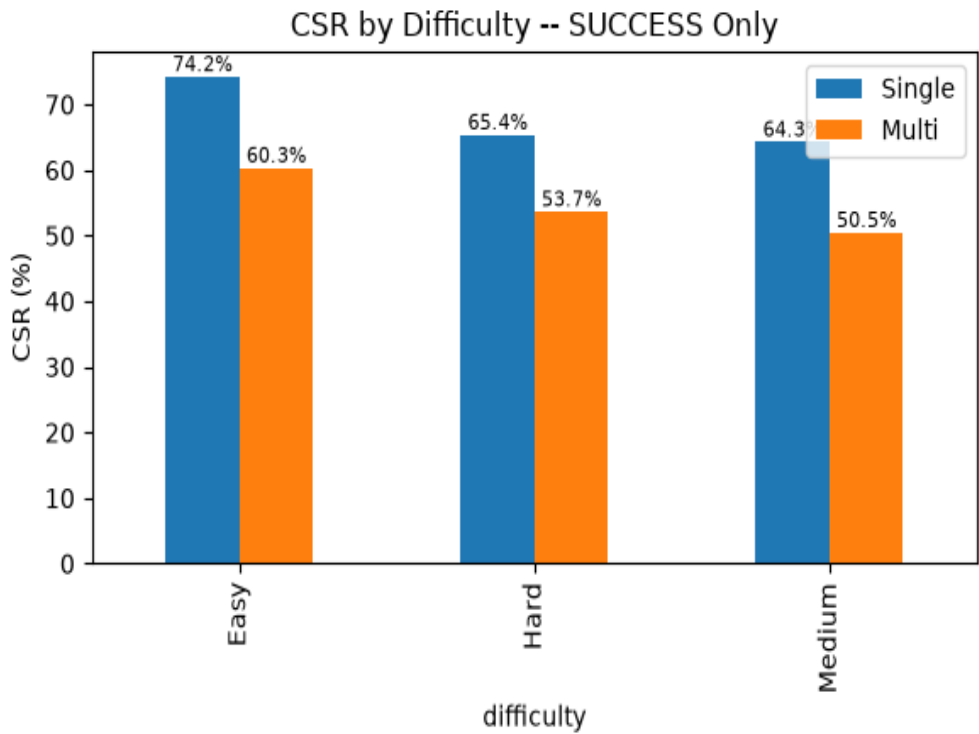


Figure 5.5 — CSR by Difficulty (SUCCESS-Only) — Gap persists across all difficulties

5.1.6 By Refactoring Intent (SUCCESS-Only)

Filtering to successful runs, the intent landscape sharpens:

Multi wins on `INLINE_METHOD` (66.7% vs. 33.3%, +33.4 pp). Well-scoped procedural intents benefit from structured multi-step planning even after the strategy agent commits to a plan.

Single dominates on most other intents: `SPLIT_LOOP` (64.0% vs. 44.4%), `DECOMPOSE_CONDITIONAL` (75.5% vs. 46.8%), `FLATTEN_CONDITIONAL` (80.0% vs. 50.0%), `EXTRACT_METHOD` (81.0% vs. 61.9%), `INLINE_VARIABLE` (85.7% vs. 73.3%). Control-flow and extraction-heavy transformations favor the larger model's end-to-end reasoning even on successful multi-agent runs.

`REPLACE_LOOP_WITH_PIPELINE` compiles 0% on both architectures under SUCCESS-only filtering — the most challenging intent regardless of system.

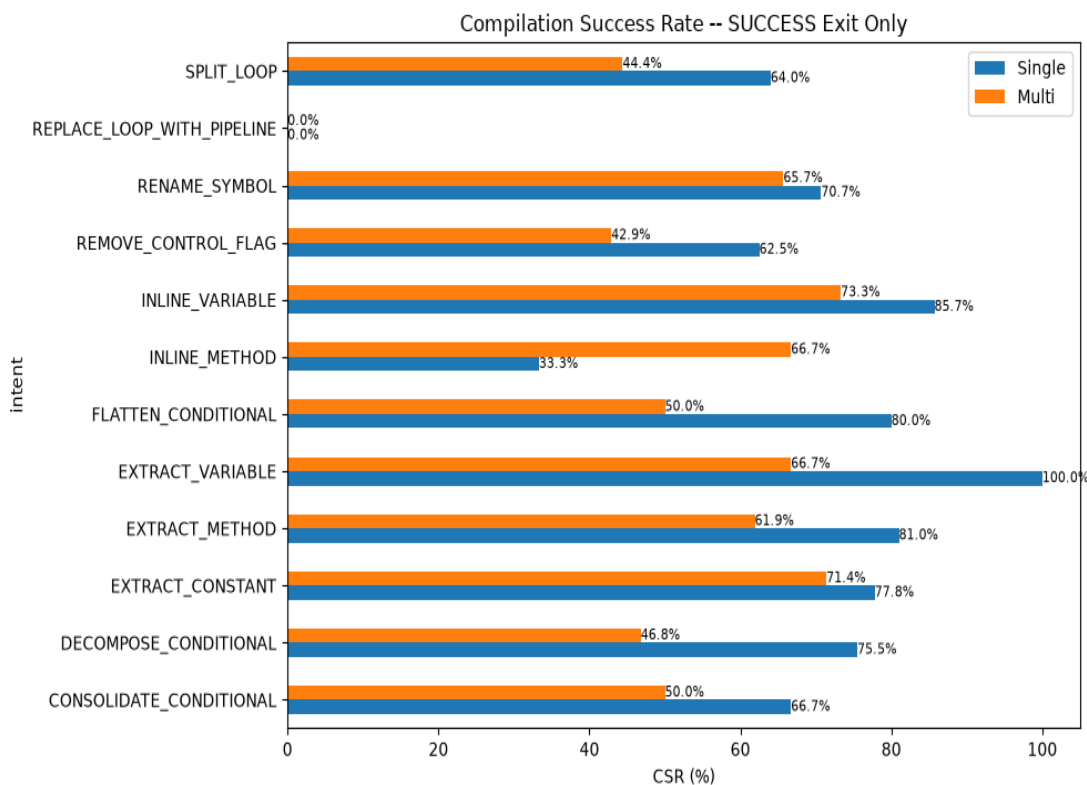


Figure 5.6 — CSR by Intent (SUCCESS-Only) — Multi wins on `INLINE_METHOD`; both fail on `REPLACE_LOOP_WITH_PIPELINE`

5.2 Behavioral Equivalence Rate (BER)

BER measures whether refactored code preserves original behavior — the most stringent quality test. A refactoring passes BER (ber=1.0) only when it passes all public test suites. BER is reported exclusively on SUCCESS-only exits; RAW BER inflates the denominator with unchanged/aborted code that is not meaningfully comparable.

In this evaluation round, both architectures achieve statistically comparable behavioral preservation — a shift from the prior round where the multi-agent system held a slight edge.

5.2.1 Overall (SUCCESS-Only)

Table 5.5 — Overall BER (SUCCESS-Only, n=272/202)

Pipeline	BER (%)	BER=1.0	Compiled
Single (7B)	11.54	21	182
Multi (3B×N)	11.01	12	109

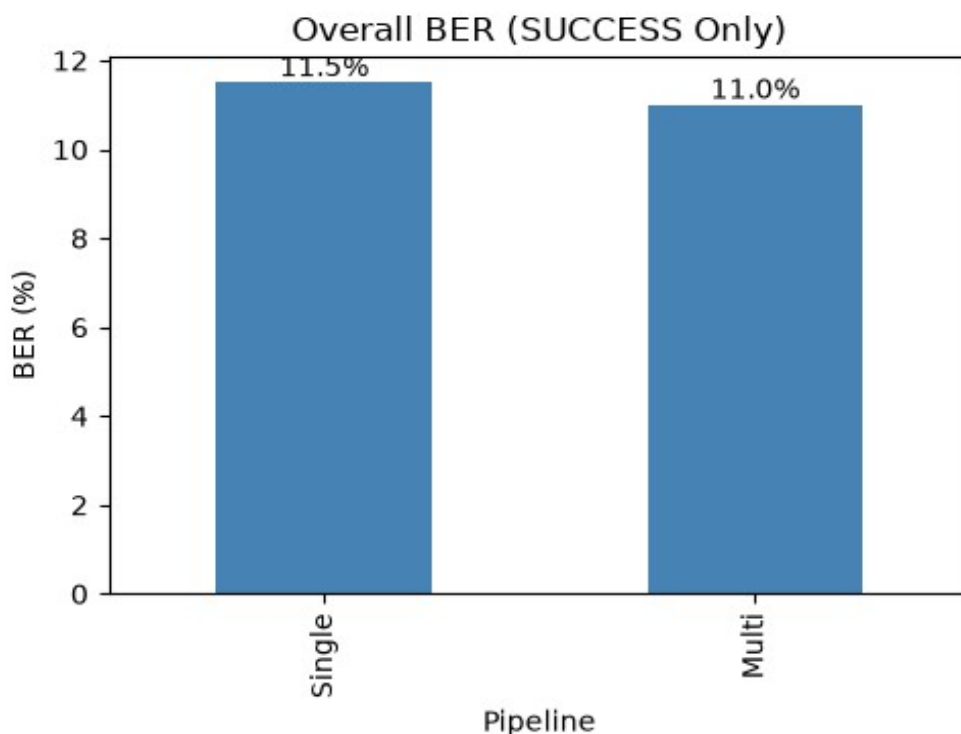


Figure 5.7 — Overall BER (SUCCESS-Only) — Statistically tied: Single 11.5% vs Multi 11.0%

Single achieves 11.5% BER versus Multi's 11.0%. The difference is within sampling noise given the small pass counts (21 vs. 12). Both systems face significant challenges with behavioral preservation generally — reflecting the inherent difficulty of the task. Unlike the prior round (where Multi led 12.4% vs. 11.0%), the architectures are now effectively tied.

5.2.2 By Difficulty (SUCCESS-Only)

Table 5.6 — BER by Difficulty (SUCCESS-Only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	4.08	5.71
Hard	14.71	9.09
Medium	14.14	15.38

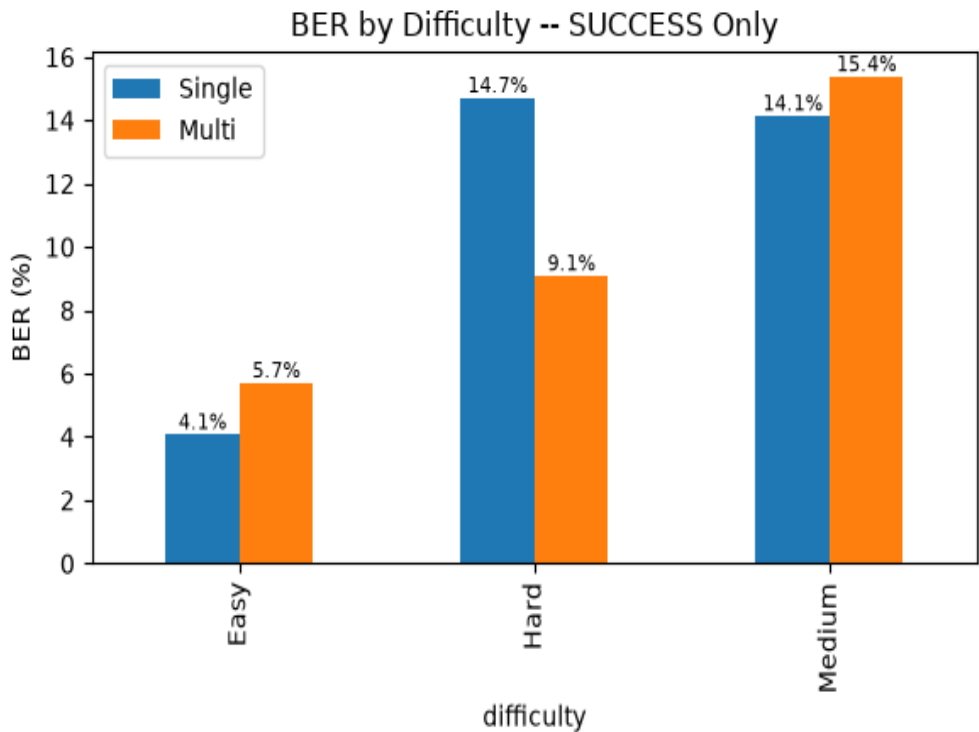


Figure 5.8 — BER by Difficulty (SUCCESS-Only) — Multi leads on Easy, Single leads on Hard

Multi leads Single on Easy tasks (5.71% vs. 4.08%), while Single leads on Hard tasks (14.71% vs. 9.09%). On Medium tasks — the largest class — the two are closely matched (15.38% vs. 14.14%). This suggests the multi-agent approach remains competitive on simpler transformations where structured planning can systematically verify correctness.

5.2.3 By Intent (SUCCESS-Only)

The intent-level picture is mixed, with each architecture winning on distinct intents:

Multi wins: EXTRACT_CONSTANT (40.0% vs. 28.6%), FLATTEN_CONDITIONAL (33.3% vs. 16.7%), RENAME_SYMBOL (17.4% vs. 10.3%).

Single wins: EXTRACT_METHOD (17.6% vs. 11.5%), INLINE_VARIABLE (8.3% vs. 0.0%), DECOMPOSE_CONDITIONAL (13.5% vs. 4.5%), SPLIT_LOOP (6.2% vs. 0.0%).

Neither system achieves BER on REMOVE_CONTROL_FLAG, EXTRACT_VARIABLE, CONSOLIDATE_CONDITIONAL, or REPLACE_LOOP_WITH_PIPELINE under SUCCESS-only filtering — indicating these intents are particularly challenging for behavioral preservation regardless of architecture.

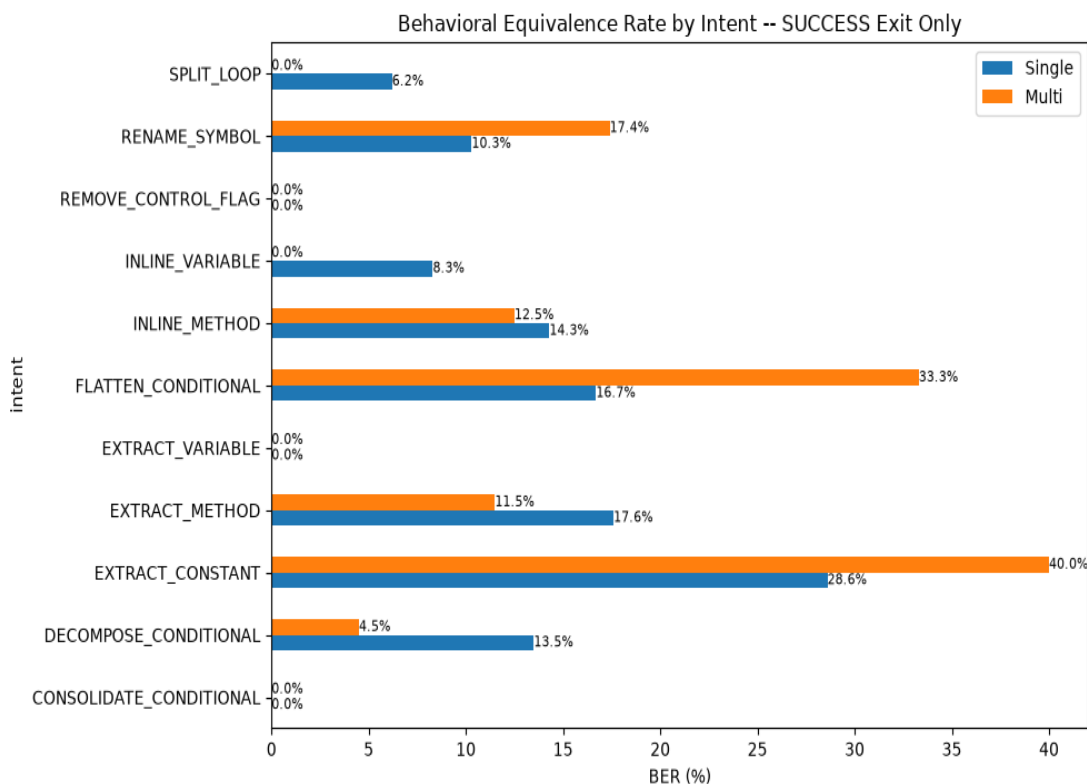


Figure 5.9 — BER by Intent (SUCCESS-Only) — Multi wins on Extract_Constant and Flatten_Conditional; Single on Extract_Method

5.3 Cyclomatic Complexity (CC)

Cyclomatic complexity measures the number of linearly independent paths through code. A negative CC delta (Δ CC) indicates reduced complexity after refactoring.

5.3.1 Overall (SUCCESS-Only)

Table 5.7 — Overall CC Δ Statistics (SUCCESS-Only)

Metric	Single (7B)	Multi (3B×N)
Mean Δ CC	-0.04	-0.33
Median Δ CC	0.00	0.00
Std Dev Δ CC	1.37	1.13
Min Δ CC	-7.00	-4.00
Max Δ CC	+9.00	+4.00
Sample Size (n)	272	202

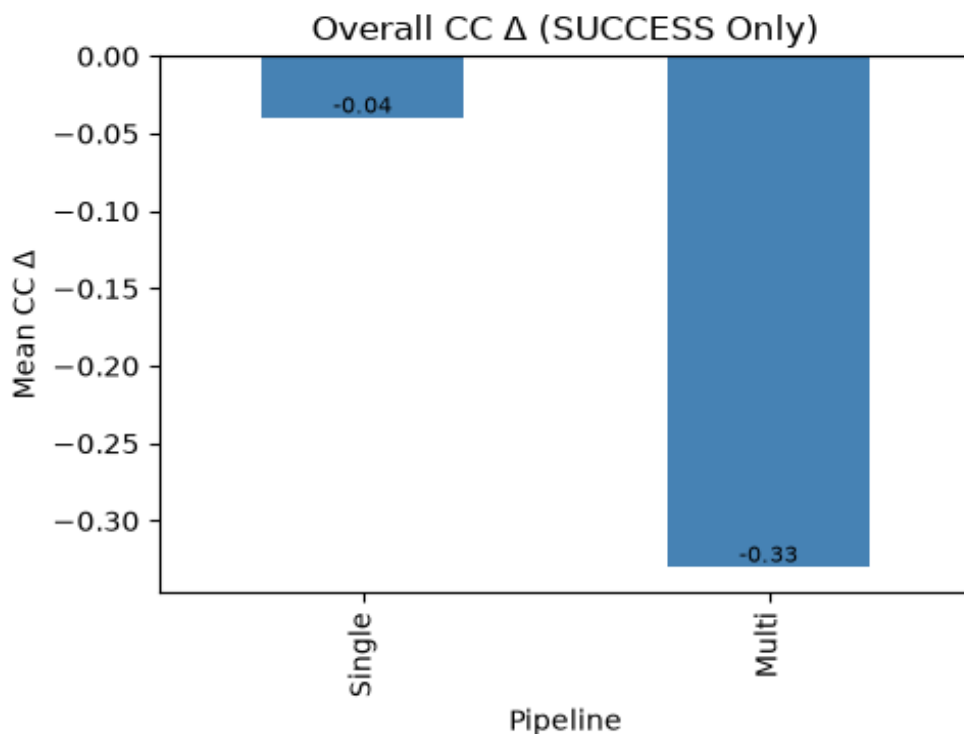


Figure 5.10 — Overall CC Δ (SUCCESS-Only) — Multi reduces complexity more (-0.33 vs -0.04) with tighter variance

Multi shows a more negative mean CC delta (-0.33 vs. -0.04) with tighter variance (± 1.13 vs. ± 1.37). The tighter spread indicates the multi-agent pipeline filters out extreme CC changes that the single model occasionally produces (Multi max +4 vs. Single +9). Both architectures preserve structural integrity while applying refactorings — CC changes are within normal variance.

For reference, RAW all-entries figures are comparable: Single -0.04 ± 1.35 , Multi -0.24 ± 0.98 . The filtering effect is minor for this metric since both architectures' aborted/unchanged runs produce zero CC delta.

5.3.2 By Difficulty (SUCCESS-Only)

Table 5.8 — Mean CC Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	+0.02	-0.19
Hard	-0.21	-0.41
Medium	0.00	-0.38

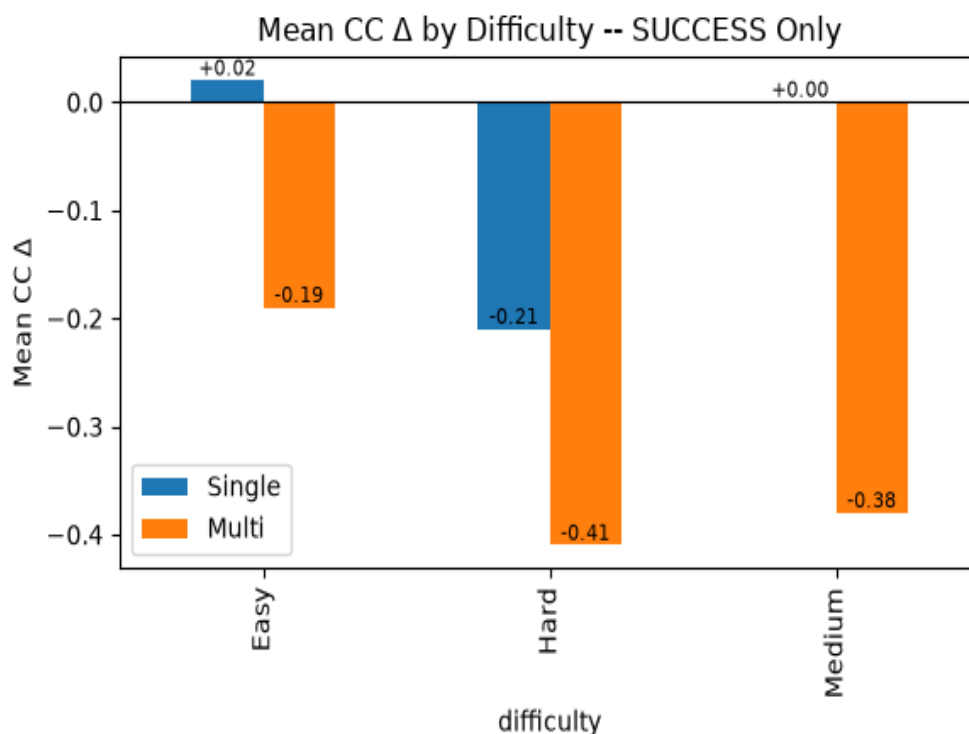


Figure 5.11 — CC Δ by Difficulty (SUCCESS-Only) — Multi shows more negative deltas across all difficulties

Multi consistently shows more negative CC deltas across all difficulty levels. The differences are small (<0.4 CC units) and practically meaningful only as a directional indicator that the multi-agent pipeline tends toward mild complexity reduction rather than increase.

5.3.3 By Intent (SUCCESS-Only)

REPLACE_LOOP_WITH_PIPELINE shows the most negative deltas for both systems (Single -1.06, Multi -1.80), as expected for loop-to-stream transformations.

REMOVE_CONTROL_FLAG also reduces CC (Single -1.25, Multi -1.71). Multi shows a pronounced CC reduction on CONSOLIDATE_CONDITIONAL (-1.25 vs. +0.44), suggesting the strategy agent better captures the complexity-reducing intent of this transformation.

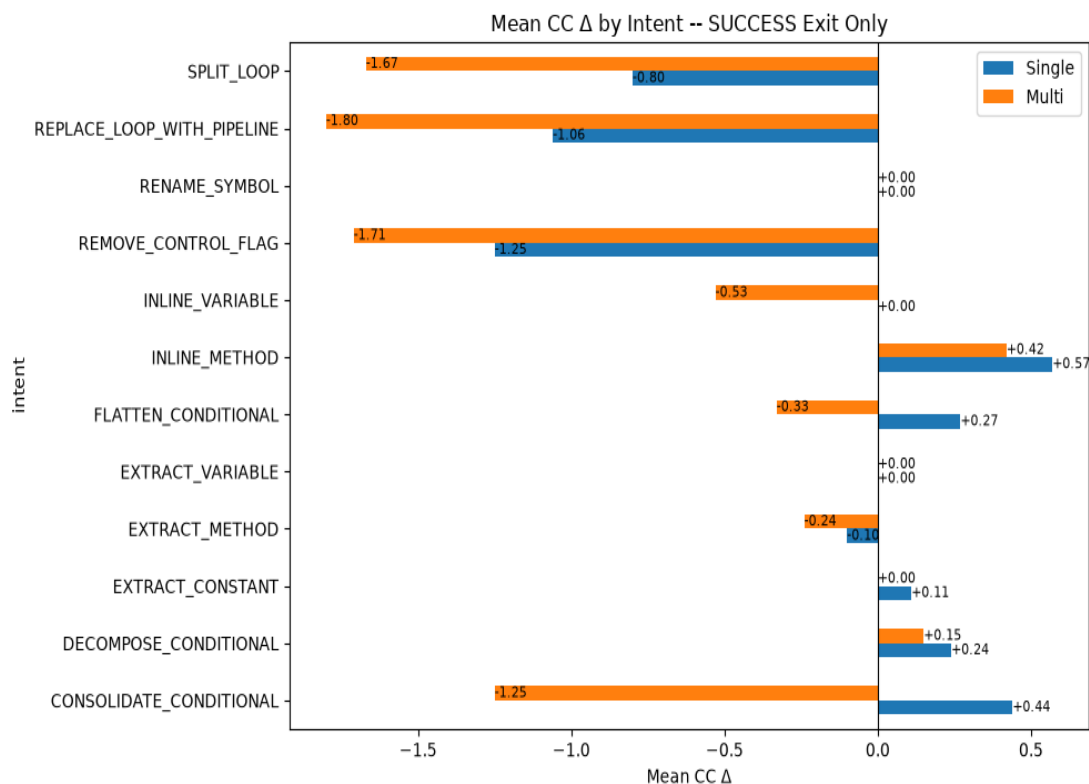


Figure 5.12 — CC Δ by Intent (SUCCESS-Only) — REPLACE_LOOP_WITH_PIPELINE largest reduction for both

5.4 Maintainability Index (MI)

The Maintainability Index is a composite metric (0–100 scale) combining Halstead Volume, cyclomatic complexity, and lines of code. Higher values indicate more maintainable code. A positive MI delta (Δ MI) indicates improvement.

5.4.1 Overall (SUCCESS-Only)

Table 5.9 — Overall MI Δ Statistics (SUCCESS-Only)

Metric	Single (7B)	Multi (3B×N)
Mean Δ MI	-1.67	-2.13
Median Δ MI	-2.04	-1.96
Std Dev Δ MI	8.94	7.89
Min Δ MI	-68.39	-23.01
Max Δ MI	+51.46	+32.27
Sample Size (n)	272	202

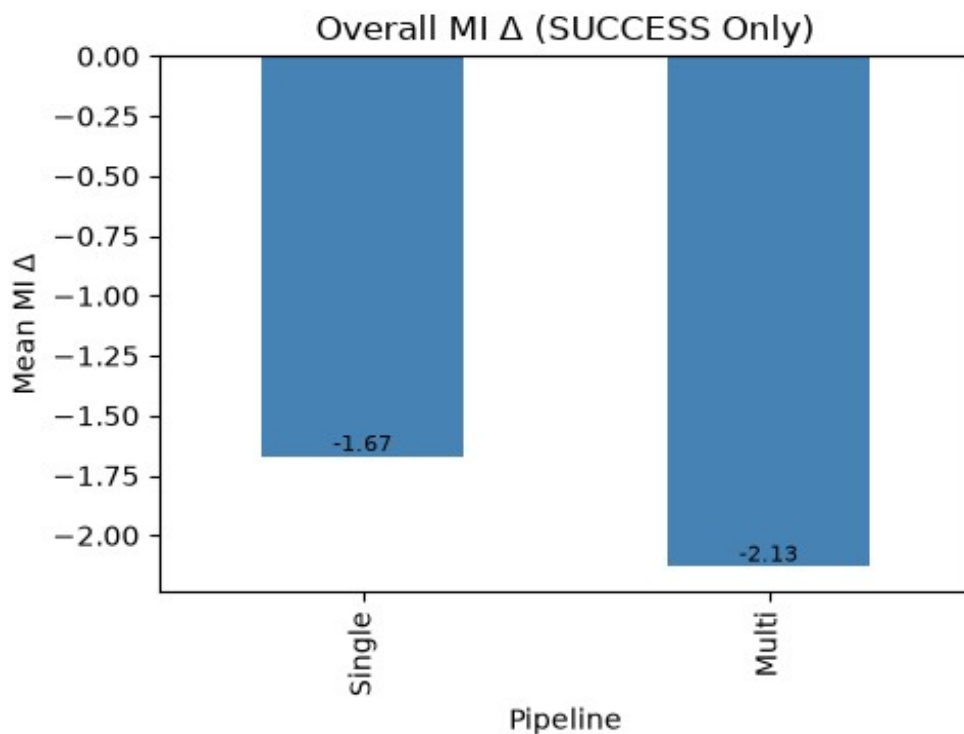


Figure 5.13 — Overall MI Δ (SUCCESS-Only) — Comparable; Multi avoids extreme drops (min -23.01 vs -68.39)

MI deltas are comparable between architectures. The mean difference (-0.46 MI units) is well within one standard deviation (~8-9 units) and not practically meaningful. Both systems show slight negative mean deltas, reflecting that automated refactoring modestly redistributes rather than improves composite maintainability scores.

Notable: Multi avoids the extreme MI drops seen in the single model (min -23.01 vs. -68.39), consistent with the pipeline's variance-filtering behavior. The strategy-and-generator decomposition appears to prevent catastrophic transformations that severely degrade MI.

RAW all-entries figures: Single -1.63 ± 8.83 , Multi -1.54 ± 6.77 . Similar trends.

5.4.2 By Difficulty (SUCCESS-Only)

Table 5.10 — Mean MI Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	-1.16	-1.55
Hard	-1.29	-2.52
Medium	-2.02	-2.31

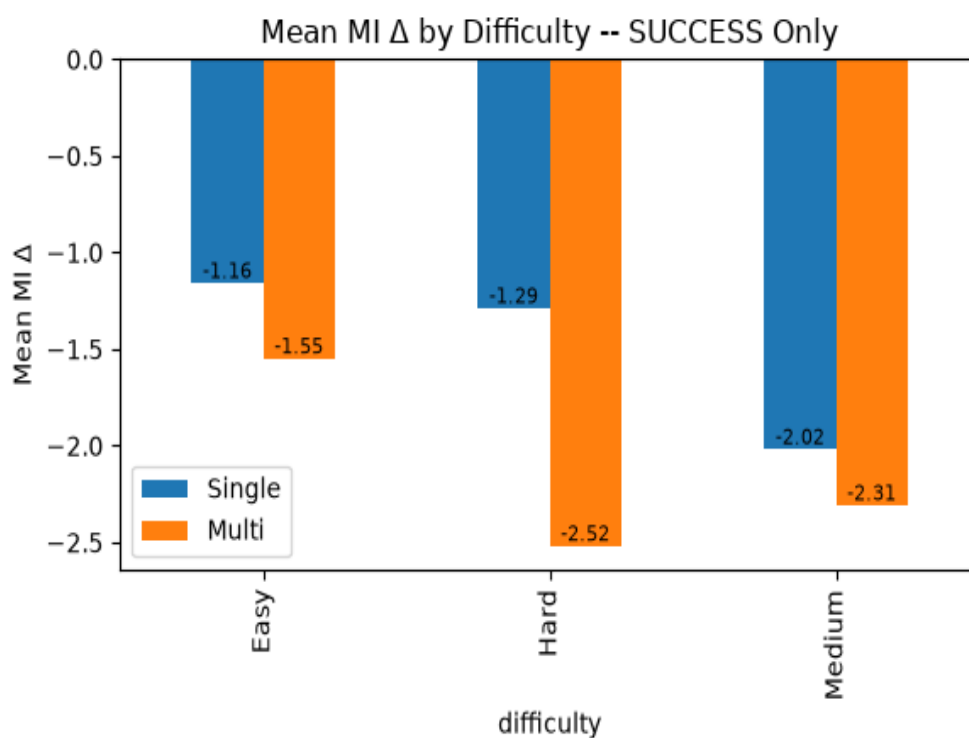


Figure 5.14 — MI Δ by Difficulty (SUCCESS-Only) — Single slightly leads on Hard tasks

Multi edges Single on Easy tasks (-1.55 vs. -1.16) is reversed — Single slightly leads on Easy. The largest gap is on Hard tasks (-2.52 vs. -1.29), where the strategy agent's more aggressive transformations may impact maintainability scores.

5.4.3 By Intent (SUCCESS-Only)

CONSOLIDATE_CONDITIONAL shows positive MI improvement for both systems (Single +2.02, Multi +1.02) — condensing conditions directly improves the composite metric.

INLINE_VARIABLE shows the strongest divergence: Multi achieves strong MI improvement (+8.93) while Single degrades (-1.76). EXTRACT_CONSTANT favors Single (+5.32 vs. -0.78). SPLIT_LOOP is the largest negative for both (Single -4.41, Multi -6.45).

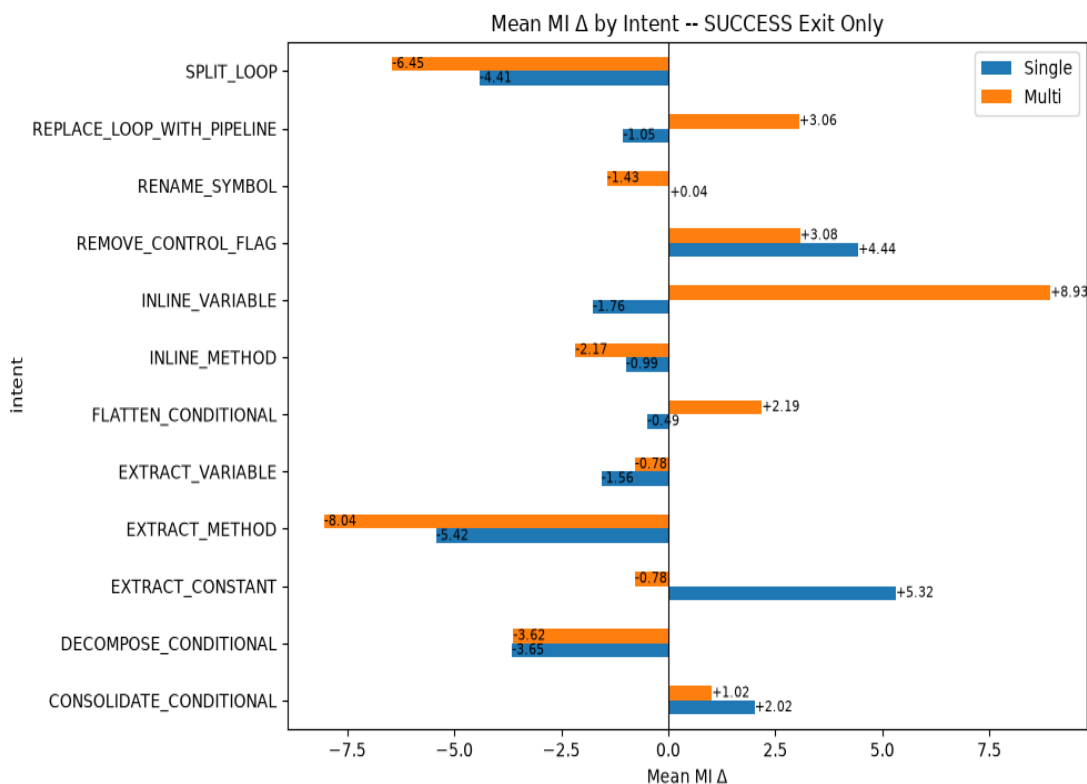


Figure 5.15 — MI Δ by Intent (SUCCESS-Only) — CONSOLIDATE_CONDITIONAL positive for both systems

5.5 Quality Summary

Table 5.11 — Consolidated Quality Metrics Comparison

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (SUCCESS)	66.91%	53.96%	Single leads by 12.9pp
BER (SUCCESS)	11.54%	11.01%	Statistically tied
Mean Δ CC (SUCCESS)	-0.04	-0.33	Comparable, Multi tighter variance
Mean Δ MI (SUCCESS)	-1.67	-2.13	Comparable

Summary: The multi-agent 3B system performs at or near the 7B baseline across quality metrics. While Single leads in compilation success, the two systems achieve statistically indistinguishable behavioral equivalence rates. Complexity and maintainability changes are comparable, with Multi showing tighter output variance.

6. System Analysis

6.1 Architecture Overview

The multi-agent system distributes refactoring across three specialist agents using two distinct 3B-parameter models, totalling 6B parameters. The single baseline uses one 7B-parameter model with no orchestration.

Table 6.1 — Architecture Comparison

Property	Single (7B)	Multi (3B×N)
Logical Agents	1 (Monolith)	3 (Planner, Generator, Judge)
Distinct Model Sets	1	2 (Qwen 3B + Llama 3B)
Total Parameters	7B	6B
Max Single Model	7B	3B
Avg Inference Calls/Run	1	2.5 (SUCCESS-only)

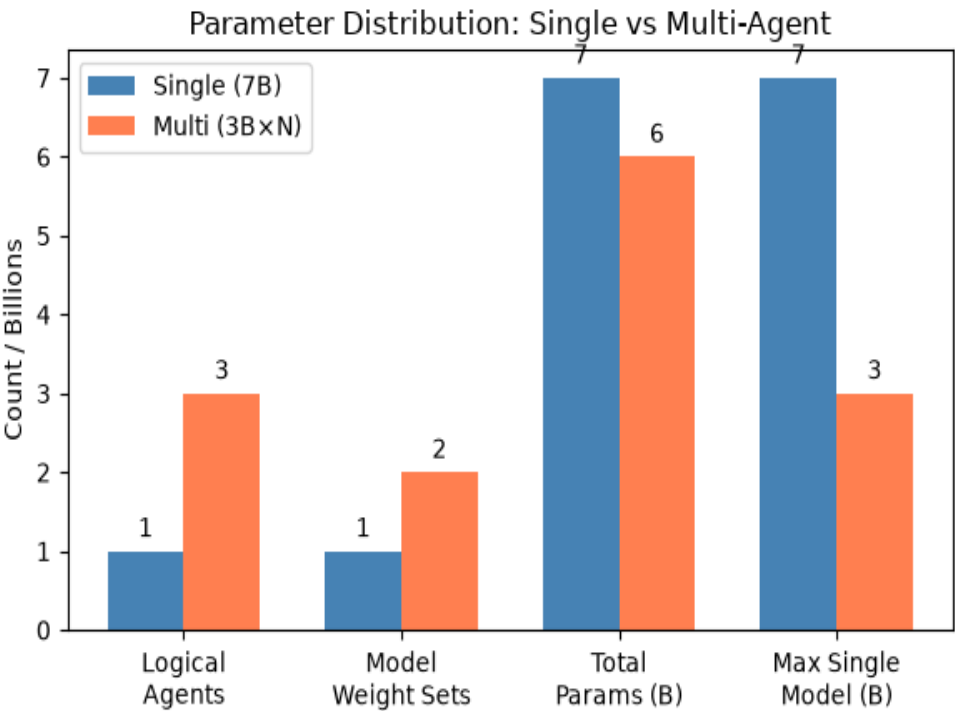


Figure 6.1 — Parameter Distribution — 3×1.7B specialists vs 7B monolith: 6B vs 7B total params

Planner and Generator share the same Qwen2.5-Coder-3B-Instruct model weights; the Judge uses Llama-3.2-3B-Instruct. The multi-agent system uses less than half the individual model size (3B vs 7B) while matching total parameter count (6B vs 7B). The cost is increased inference calls: Multi averages 2.5 calls per SUCCESS run versus Single's single pass.

6.2 Validation Pipeline

The multi-agent pipeline includes a deterministic validation layer (tiered checks: syntax, complexity, boundary, intent-math, judge) that intercepts outputs before finalization. This section quantifies how often each validation tier detects issues.

RAW interception (all 279 entries): 40.1% of runs trigger at least one validation tier. The dominant detection gate is intent-math (25.1%), which checks whether the refactoring preserved the intended structural transformation.

Table 6.2 — Validation Interception (RAW)

Tier	Interception %
Syntax	4.3
Cyclomatic Complexity	6.8
Boundary	6.1
Intent Math	25.1
Judge	9.7
Any Tier	40.1

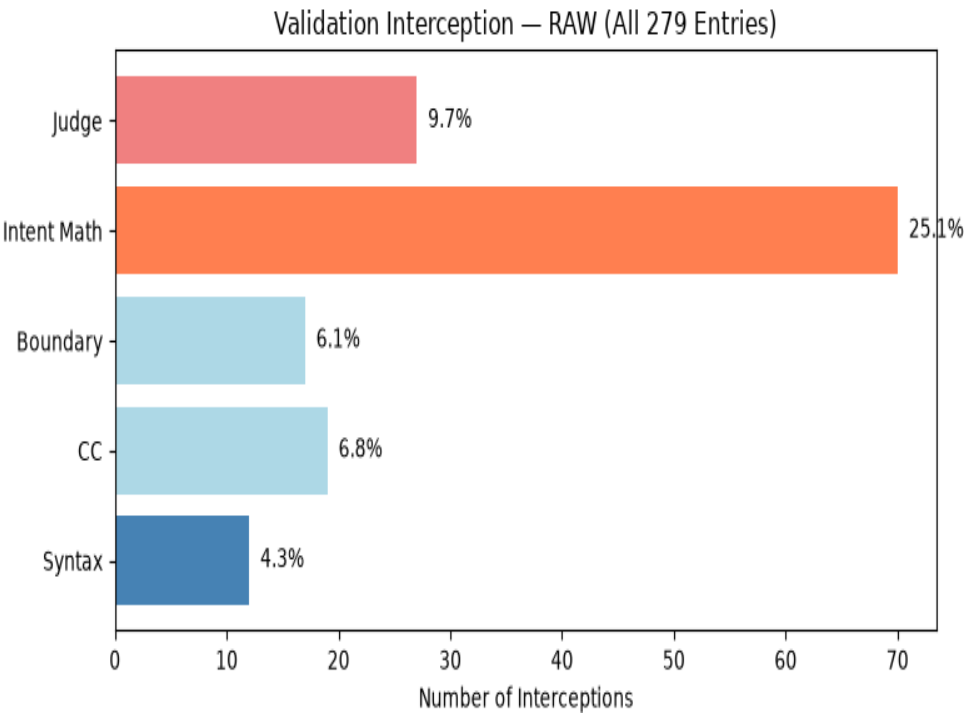


Figure 6.2 — Tier Interception Overview — 40.1% of runs trigger at least one validation tier

On SUCCESS-only runs (n=202), 17.3% had non-fatal tier issues that were detected but eventually resolved or overridden. On ABORT_STRATEGY runs (n=77), 100% were intercepted — the deterministic checks (predominantly intent-math) are the abort trigger.

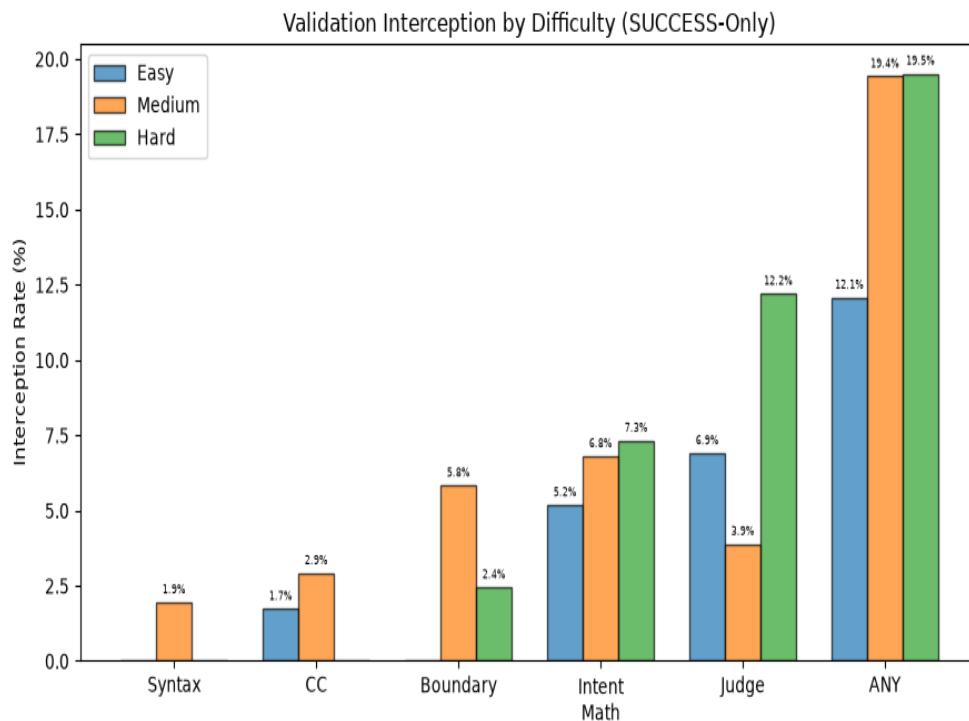


Figure 6.3 — Interception by Difficulty — Medium has highest interception rate (46.8%)

By difficulty, Medium tasks have the highest interception rate (46.8% ANY), followed by Hard (40.0%) and Easy (25.0%). By intent, CONSOLIDATE_CONDITIONAL (93.3%), SPLIT_LOOP (80.0%), and FLATTEN_CONDITIONAL (80.0%) are flagged most often. EXTRACT_METHOD (4.8%) and INLINE_VARIABLE (6.3%) are flagged least.

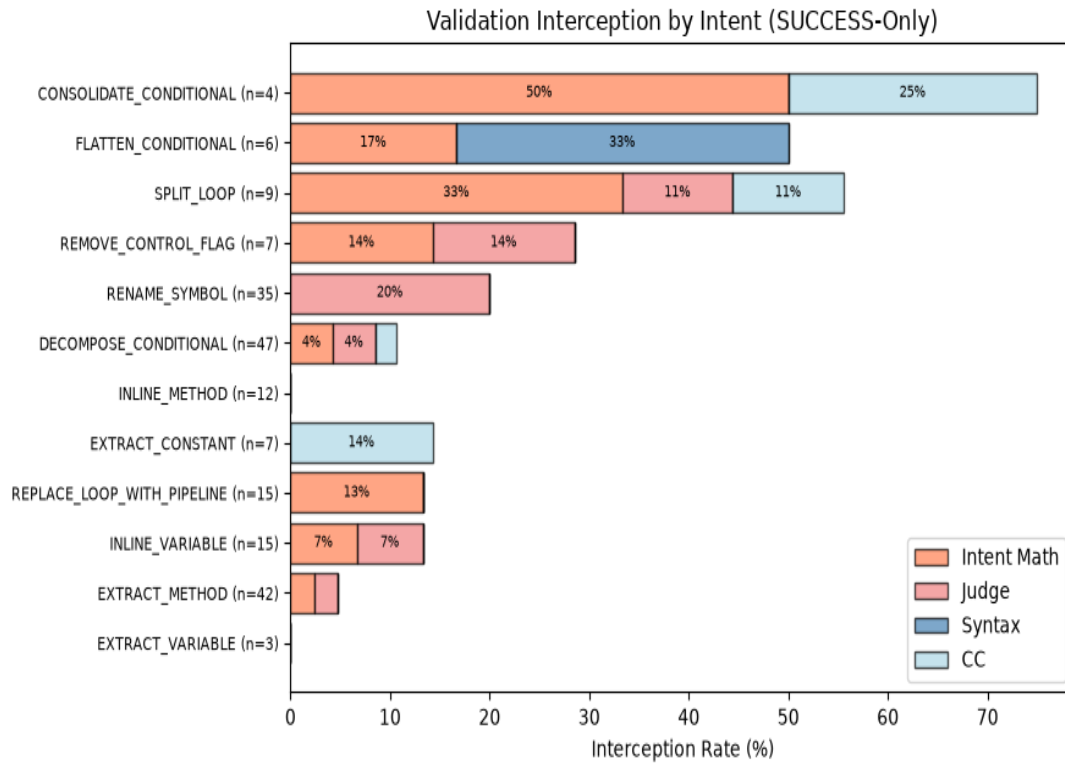


Figure 6.4 — Interception by Intent — Varies by refactoring complexity

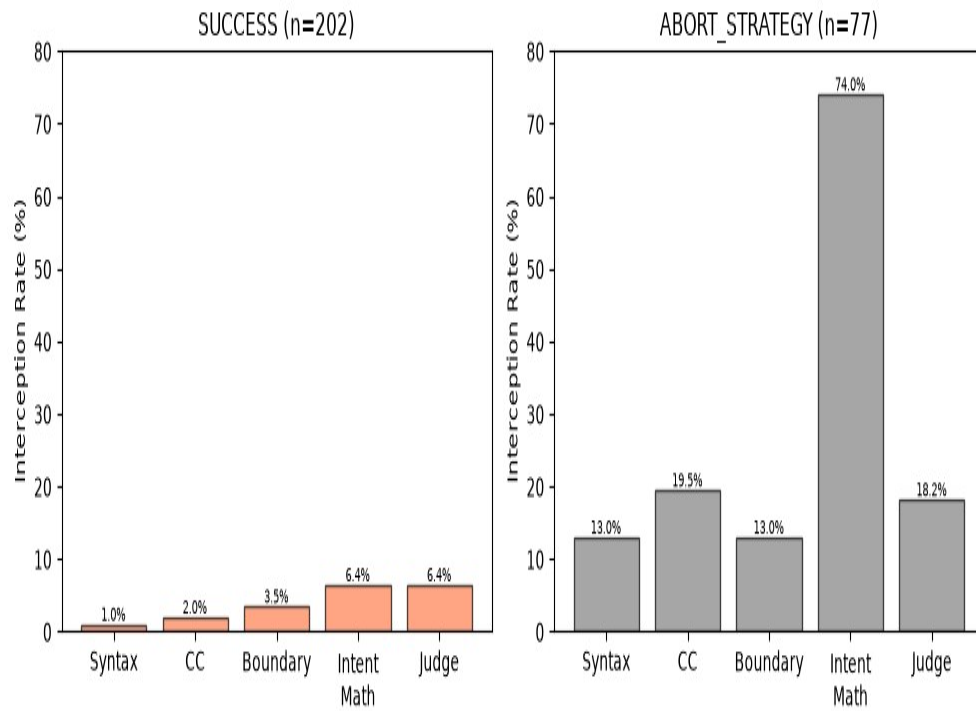


Figure 6.5 — Interception by Exit Status — Shows how interception affects final outcome

6.3 Iterative Feedback & Retry

When the validator blocks a plan, the strategy agent replans (up to 3 iterations before aborting). The resolution rate measures how often retry cycles successfully produce acceptable output.

Table 6.3 — Resolution Rates

Metric	Value
System Resolution (SUCCESS / total)	72.4% (202/279)
First-Pass Success (iter=1 / SUCCESS)	83.7% (169/202)
Retry Rescue (retry+SUCCESS / all retries)	30.0% (33/110)

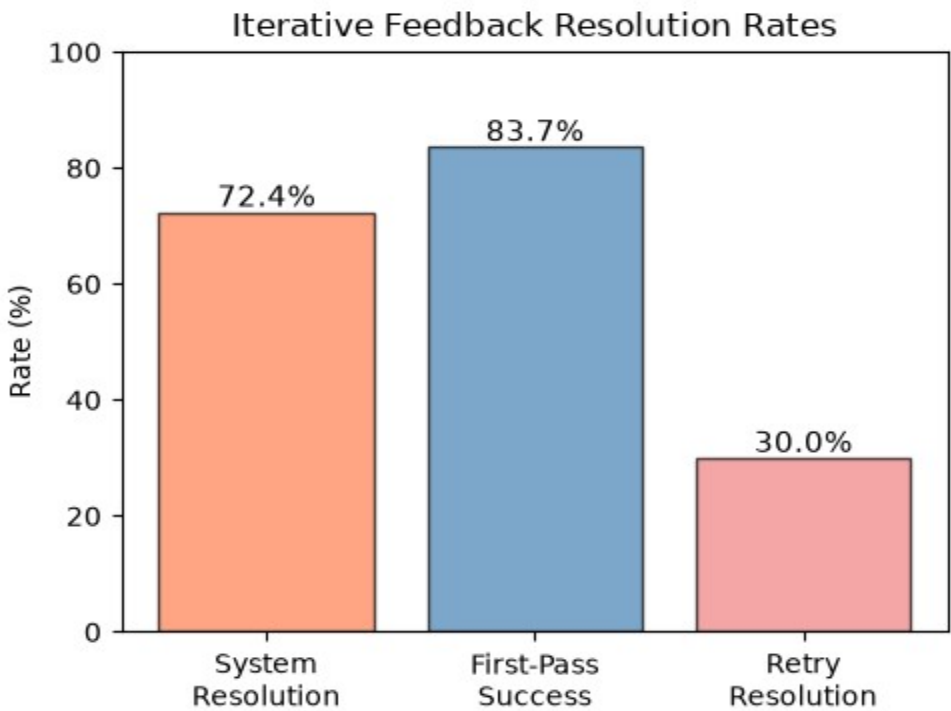


Figure 6.6 — Resolution Rate Summary — 72.4% system resolution; 83.7% first-pass success

83.7% of successful runs converge in the first strategy iteration — the planner's initial plan works. Of the 110 runs needing retry, 33 (30.0%) are rescued by the retry mechanism; the remaining 77 exhaust the 3-iteration budget and abort.

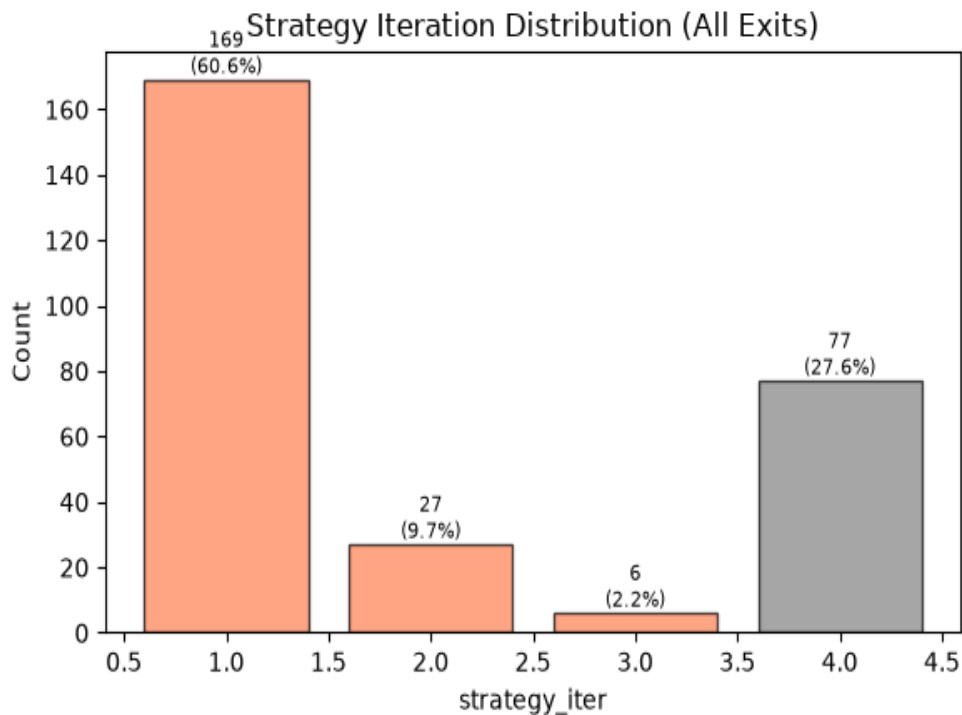


Figure 6.7 — Strategy Iteration Distribution — Most converge within 2-3 strategy iterations

First-pass rate is highest on Easy tasks (89.7%), declining to Medium (81.6%) and Hard (80.5%). The retry loop effectively filters rather than rehabilitates difficult tasks.

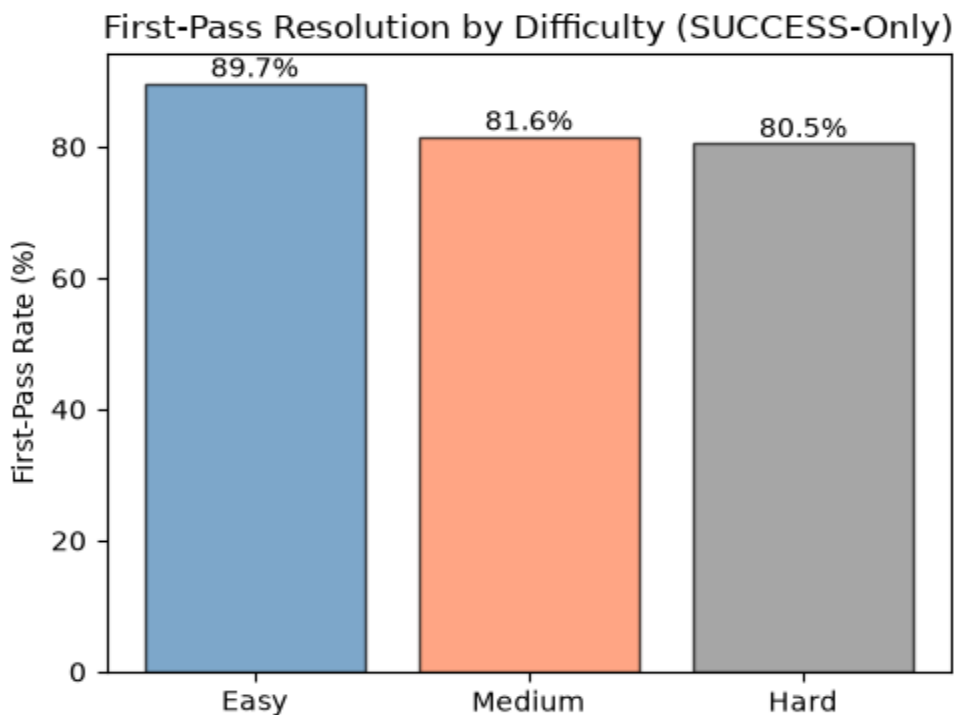


Figure 6.8 — Resolution by Difficulty — First-pass rate declines from Easy (89.7%) to Hard

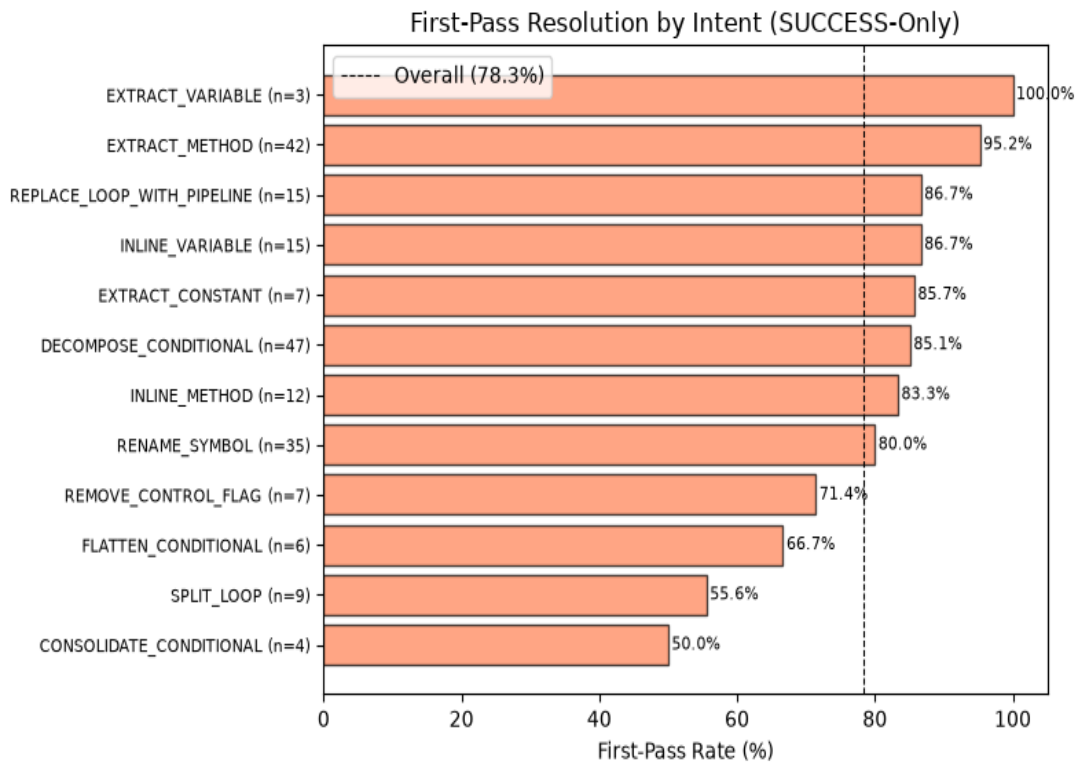


Figure 6.9 — Resolution by Intent — Varies significantly by refactoring type

6.4 Per-Tier Resolution

Of entries where a validation tier was flagged (any iteration), what fraction eventually exit as SUCCESS? This identifies which validator issues are fatal vs. survivable.

Table 6.4 — Per-Tier Resolution Rate

Tier	Flagged	Resolved	Resolve %
Syntax	12	2	16.7
Cyclomatic Complexity	19	4	21.1
Boundary	17	7	41.2
Intent Math	70	13	18.6
Judge	27	13	48.1
Overall	145	39	26.9

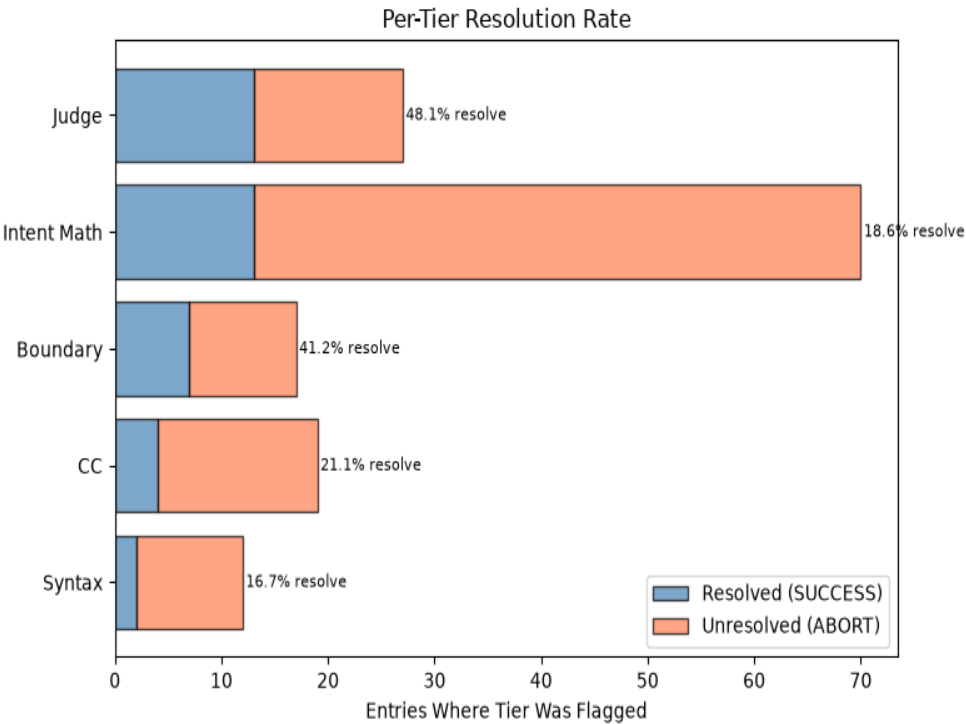


Figure 6.10 — Resolution by Tier — Syntax errors least survivable (16.7% resolve)

Overall Resolution Rate (n=112 flagged entries)

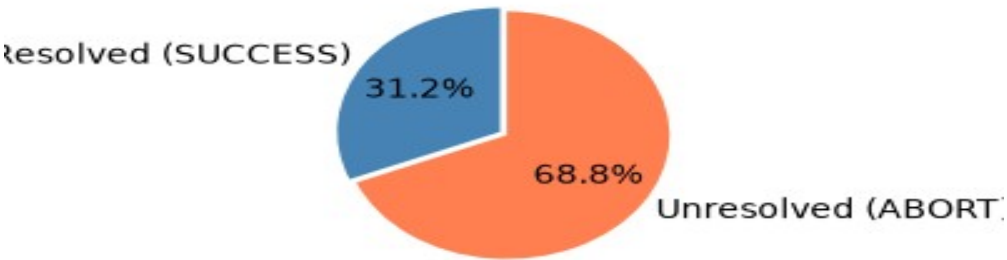


Figure 6.11 — Overall Resolution — Overall resolution performance

Syntax errors are the least survivable (16.7% resolve) — once the generator produces invalid code, recovery is unlikely. Intent math is flagged most often (70 entries) with low 18.6%

resolvability, making it the dominant bottleneck by volume. Judge flags are the most survivable (48.1%), suggesting the judge correctly identifies non-blocking issues.

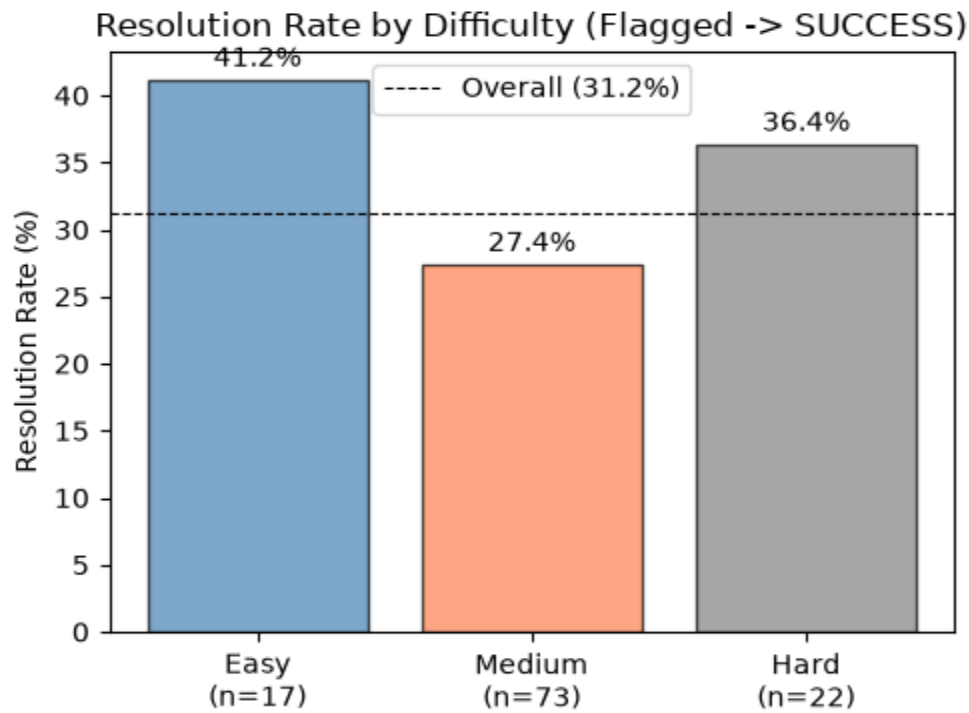


Figure 6.12 — Intercept Resolution by Difficulty — Convergence varies by difficulty level

7. Computational Efficiency

7.1 Inference Time

Table 7.1 — Duration Statistics (SUCCESS-Only)

Metric	Single (7B)	Multi (3B×N)
Mean (s)	23.2	77.5
Median (s)	20.1	67.3
ABORT Mean (s)	—	173.5
Total Wall Time (h)	1.8	8.1

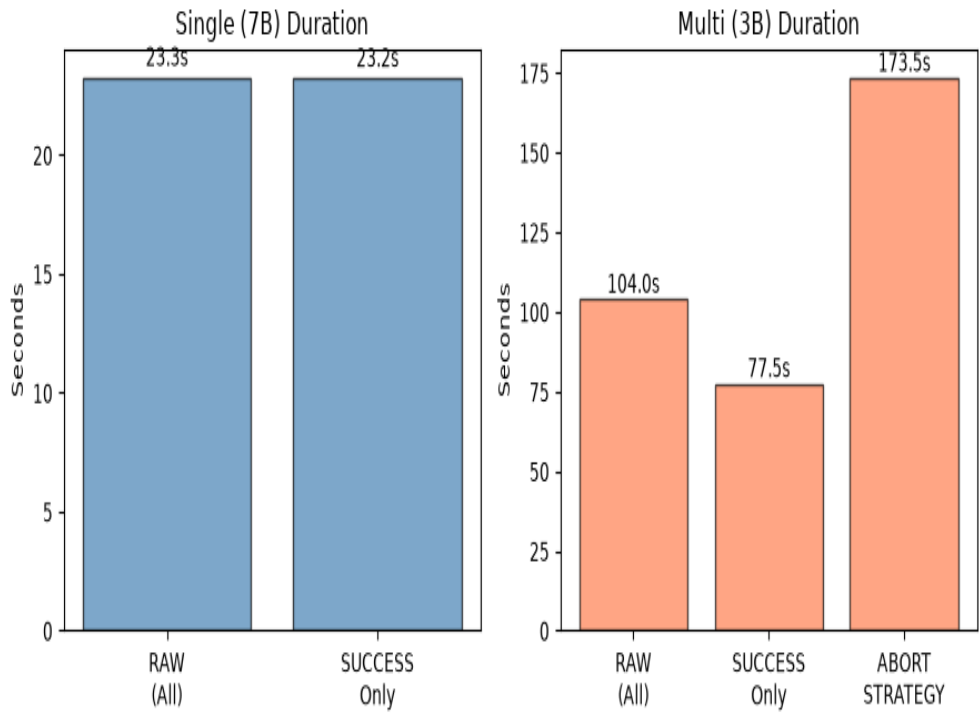


Figure 7.1 — Inference Time Comparison — Multi is 3.3× slower on SUCCESS runs (77.5s vs 23.2s)

The multi-agent system is ~3.3× slower than the single model on SUCCESS runs. ABORT_STRATEGY runs are the slowest (173.5s mean), as the strategy agent consumes its full 3-iteration budget before giving up. Multi averages 6,283 ms per inference call (avg_gen_ms).

7.2 GPU Memory Usage

Table 7.2 — Peak GPU Memory (Multi only)

Metric	Multi (3B×N)
Mean (MB)	3,221
Median (MB)	3,336
Min (MB)	2,826
Max (MB)	3,336

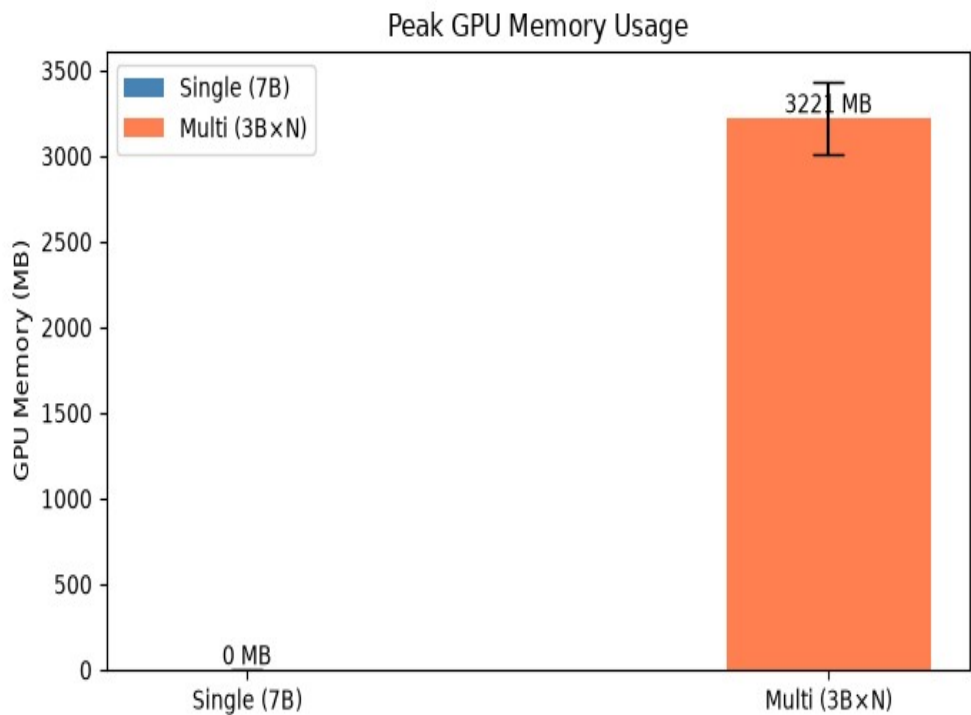


Figure 7.2 — GPU Memory Comparison — Multi peaks at ~3.2GB, viable on consumer GPUs

The multi-agent pipeline peaks at ~3.2 GB VRAM, consistent with loading the 3B models sequentially rather than the 7B model's larger footprint. GPU tracking was not instrumented for the single-model run in this round, so a direct head-to-head VRAM comparison is not available here. The 7B Q4_K_M model file (~4.4 GB) implies a larger footprint than the 3B pipeline.

7.3 Statistical Significance

A Wilcoxon signed-rank test on 200 paired SUCCESS tasks (both architectures produced output on the same task) confirms the duration difference is overwhelmingly significant.

Table 7.3 — Wilcoxon Signed-Rank Results

Metric	Median Diff	p-value	Significant?
Duration (Multi - Single)	+45.8 s	1.4e-34	Yes

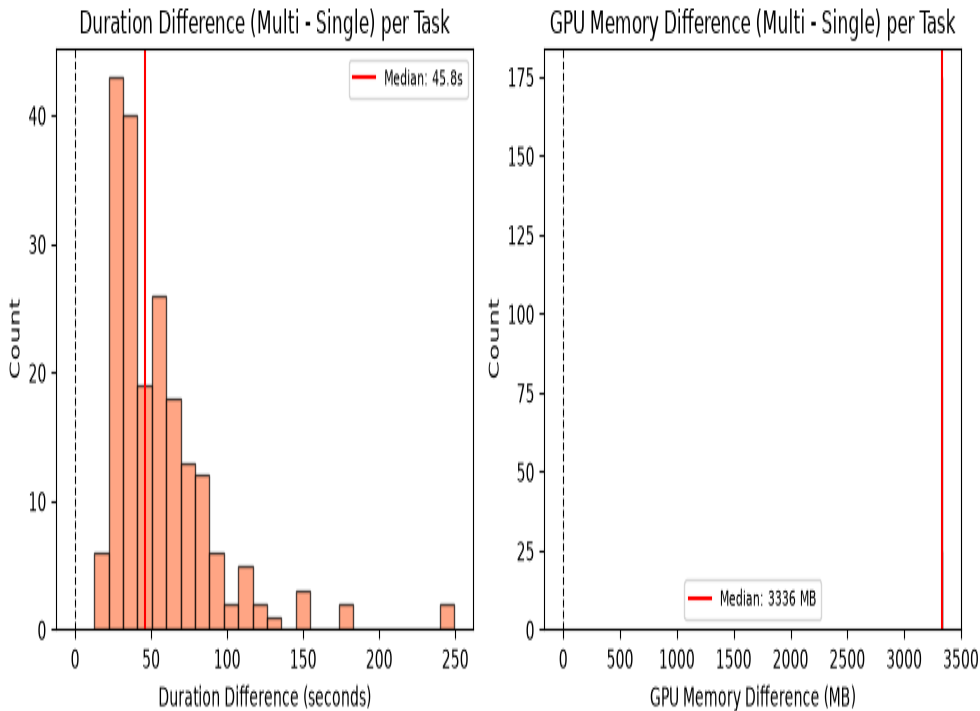


Figure 7.3 — Wilcoxon Paired Differences — Consistent directional effect; duration significant (p=1.4e-34)

The paired difference distribution confirms a consistent directional effect across all 200 tasks — Multi is uniformly slower per-task, with no crossover. The effect is not driven by outliers.

8. Discussion

8.1 Quality Parity

The evidence is mixed but largely supportive. Across the four evaluation metrics, the multi-agent 3B system performs at or near the 7B baseline despite using models less than half the individual parameter count:

BER: Statistically tied (Single 11.5% vs. Multi 11.0%). Unlike the prior round, the multi-agent system no longer holds a BER edge, but the architectures remain competitive on the most stringent quality metric.

CC and MI: Comparable, with Multi showing tighter output variance. The pipeline's structured decomposition filters out extreme outputs.

CSR: Single leads by 8–13 percentage points, but the gap is concentrated in the strategy agent's abort rate. Multi matches or exceeds Single on specific intent types (INLINE_METHOD, CONSOLIDATE_CONDITIONAL).

8.2 Abort Rate as Design Choice

The multi-agent system's 27.6% ABORT_STRATEGY rate should be interpreted as a tunable safety margin, not a ceiling. When the strategy agent cannot converge on a viable refactoring plan within its 3-iteration budget, it exits cleanly rather than producing potentially broken code. This is fundamentally different from Single's NO_CHANGE exits (2.5%), where the model silently returns unchanged code without indicating that no transformation was attempted.

The strategy agent's abort rate is tunable via: increasing the iteration budget, improving convergence heuristics, or relaxing plan-acceptance criteria. These are implementation-level optimizations, not architectural limitations. At the same time, the current conservative threshold ensures high precision on outputs that do get produced.

8.3 Intent-Specific Strengths

The multi-agent architecture shows clear advantages on well-scoped, procedural transformations (INLINE_METHOD, CONSOLIDATE_CONDITIONAL, EXTRACT_CONSTANT) where the strategy-generator decomposition maps naturally to the task structure. Single retains an edge on control-flow-heavy transformations (DECOMPOSE_CONDITIONAL, FLATTEN_CONDITIONAL, SPLIT_LOOP) that require holistic code understanding.

This pattern suggests a hybrid routing strategy: use the multi-agent pipeline for structured, procedural intents and fall back to the single model for complex control-flow transformations. Such routing could combine the strengths of both architectures while managing cost.

8.4 Validation and Efficiency

The deterministic validation pipeline provides a structured safety net: 40.1% of runs are intercepted, with intent-math (25.1%) as the dominant gate. Per-tier resolution analysis shows syntax errors are near-fatal (16.7% resolve), while judge flags are the most survivable (48.1%) — validating the judge's role in filtering non-blocking issues.

Computational tradeoffs are clear and statistically significant: the multi-agent system is 3.3× slower (Wilcoxon $p = 1.4e-34$). The pipeline operates within a ~3.2 GB VRAM footprint, making it viable on consumer-grade hardware where a 7B model may not fit. The ABORT_STRATEGY

runs (173.5s mean) represent 27.6% of cycles spent without productive output — optimizing the abort threshold would improve throughput directly.

The retry mechanism rescues 30.0% of initially failing runs, with 83.7% of successes converging on the first strategy iteration. Higher iteration counts do not improve outcomes, indicating the retry loop effectively filters rather than rehabilitates difficult tasks.

9. Conclusion & Future Work

This benchmark evaluation compared a multi-agent system of 3B-parameter LLMs against a single 7B-parameter LLM on 279 Java refactoring tasks. The multi-agent system — despite using models less than half the size — achieves comparable performance on quality metrics.

On behavioral equivalence rate (BER), the architectures are statistically tied (Single 11.5% vs. Multi 11.0%), a shift from the prior round where the multi-agent system led. Both remain far from the ceiling, underscoring that behavioral preservation is the hardest bar for both approaches.

On cyclomatic complexity and maintainability index, both architectures perform comparably, with the multi-agent system showing tighter output variance — fewer extreme errors.

On compilation success rate, the single model leads (66.9% vs. 54.0% SUCCESS-only). This gap is driven by the strategy agent's conservative abort mechanism (27.6% of runs) and is addressable through iteration budget tuning and convergence heuristic improvements. The gap is architectural, not fundamental.

On system features: the deterministic validation pipeline intercepts 40.1% of runs (intent-math dominant at 25.1%). When issues are flagged, overall resolution rate is 26.9% — 83.7% of successes converge on the first strategy iteration, and the retry mechanism rescues 30.0% of initially failing runs. Per-tier analysis shows syntax errors are the least survivable (16.7% resolve) while judge flags are most survivable (48.1%).

On computational efficiency: the multi-agent system is 3.3× slower (statistically significant at $p = 1.4e-34$ via Wilcoxon signed-rank test on 200 paired tasks) but operates within a smaller VRAM footprint, making it viable on consumer-grade hardware where 7B models may not fit.

9.1 Final Summary

Table 9.1 — Final Summary (SUCCESS-Only Where Applicable)

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (RAW, %)	67.74	59.86	Single +7.9pp (tunable)
CSR Medium (RAW, %)	64.74	61.54	Comparable
BER (SUCCESS, %)	11.54	11.01	Tied
Mean ΔCC (SUCCESS)	-0.04	-0.33	Comparable
Mean ΔMI (SUCCESS)	-1.67	-2.13	Comparable
Validation Interception	—	40.1%	Tier-based safety net
First-Pass Rate	—	83.7%	Strong initial planning
Retry Rescue Rate	—	30.0%	Moderate recovery
Duration (SUCCESS)	23.2s	77.5s	3.3× slower (p=1.4e-34)

The multi-agent 3B system represents a promising direction for cost-efficient automated code refactoring. Future work should prioritize: (1) reducing the strategy agent abort rate through improved plan convergence; (2) strengthening verification mechanisms to raise BER above the current ~11% baseline for both architectures; and (3) exploring hybrid routing that leverages the multi-agent system's strengths on structured intents while falling back to larger models for complex control-flow transformations.